Estruturas de Dados Básicas
Deque & Lista

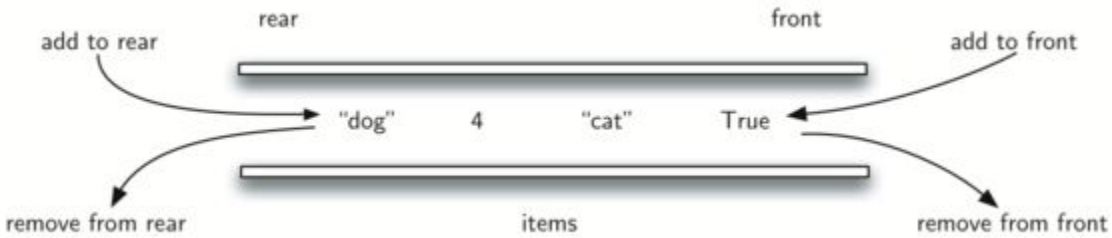
Estruturas de Dados



Universidade de Brasília
Departamento de Ciência da Computação

Deque

Um deque, também conhecido como fila de duas extremidades, é uma coleção ordenada de itens. Tem duas extremidades, uma é o início (front) e uma é o fim (rear), e os itens permanecem posicionados na coleção.



Tipo Abstrato de Dados Deque

- `Deque()` cria um novo deque que está vazio.
- `addFront(item)` insere um novo `item` no início da deque.
- `addRear(item)` insere um novo `item` no fim da deque.
- `removeFront()` remove o item do início da deque.
- `removeRear()` remove o item do fim da deque.
- `isEmpty()` testa se o deque está vazio.
- `size()` retorna o número de itens no deque.

Tipo Abstrato de Dados Deque

```
class Deque:
    def __init__(self):
        self.items = []

    def isEmpty(self):
        return self.items == []

    def addFront(self, item):
        self.items.append(item)

    def addRear(self, item):
        self.items.insert(0,item)

    def removeFront(self):
        return self.items.pop()

    def removeRear(self):
        return self.items.pop(0)

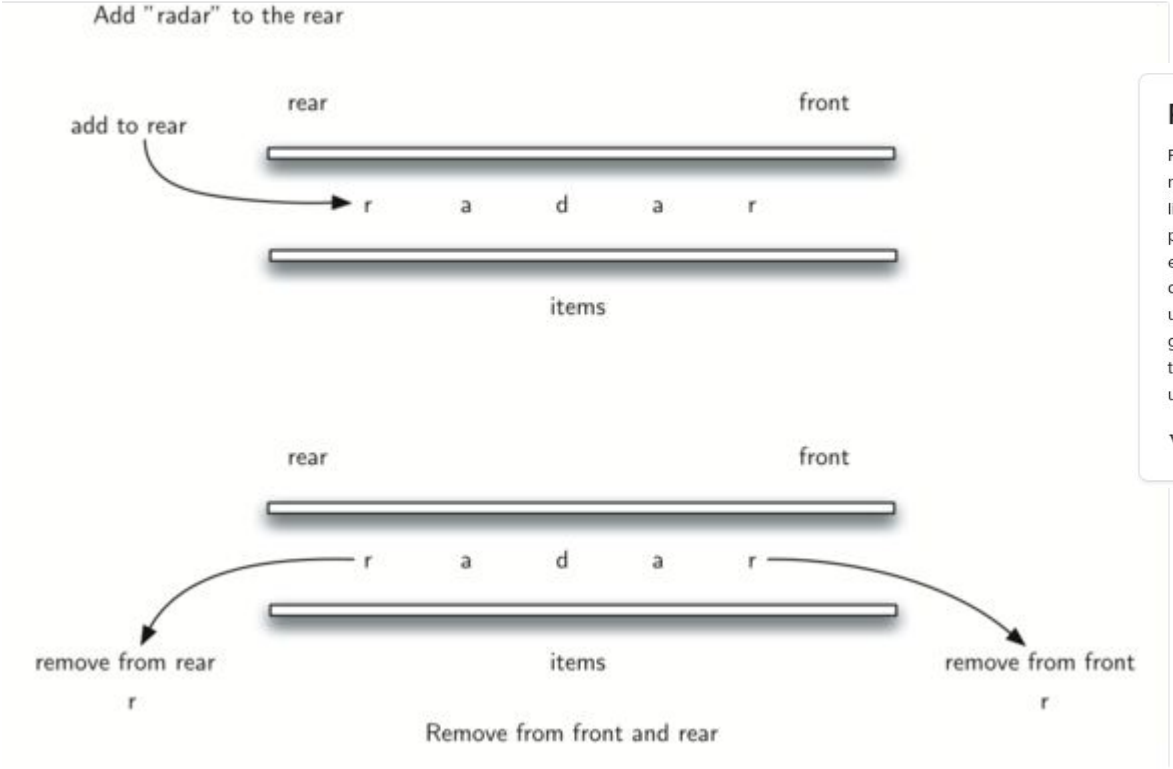
    def size(self):
         return len(self.items)
```

```
from pythonds.basic.deque import Deque
from collections import deque
```

Insere e remove no final da lista (rear) - O(1)

Insere e remove no início da lista (front) - O(n)

Verificador de Palíndromos



Palíndromo

S	A	T	O	R
A	R	E	P	O
T	E	N	E	T
O	P	E	R	A
R	O	T	A	S

Palíndromo é uma palavra, frase ou número que permanece igual quando lida de trás para diante. Por extensão, palíndromo é qualquer série de elementos com simetria linear, ou seja, que apresenta a mesma sequência de unidades nos dois sentidos. A palavra "palíndromo" vem do grego palin + dromos - que corre em sentido inverso. Mas o termo foi cunhado em inglês, no século XVII, e os primeiros a utilizá-lo foram os escritores Henry Peacham e Ben Jonson.

W More at Wikipedia (PT)

Verificador de Palíndromos

```
from pythonds.basic.deque import Deque

def palchecker(aString):
    chardeque = Deque()

    for ch in aString:
        chardeque.addRear(ch)

    stillEqual = True

    while chardeque.size() > 1 and stillEqual:
        first = chardeque.removeFront()
        last = chardeque.removeRear()
        if first != last:
            stillEqual = False

    return stillEqual
```

Lista

Uma lista é uma coleção de itens em que cada item tem uma posição relativa em relação aos outros. Mais especificamente, nos referimos a este tipo de lista como uma lista desordenada.

```
l = []
l = list()
l = [54, 26, 93, 17, 77, 31]
```

Tipo Abstrato de Dados Lista

- `List()` cria uma nova lista que está vazia.
- `add(item)` insere um `item` novo à lista.
- remove(item) remove um `item` da lista.
- `search(item)` procura o `item` na lista.
- `isEmpty()` verifica se a lista está vazia.
- `append(item)` adiciona um novo `item` ao final da lista.
- `index(item)` retorna a posição do `item` na lista.
- `insert(pos, item)` adiciona um novo `item` à lista na posição `pos`.
- `pop()` remove e retorna o último item da lista.
- `pop(pos)` remove e retorna o item na posição `pos`.

Implementando Listas Ligadas

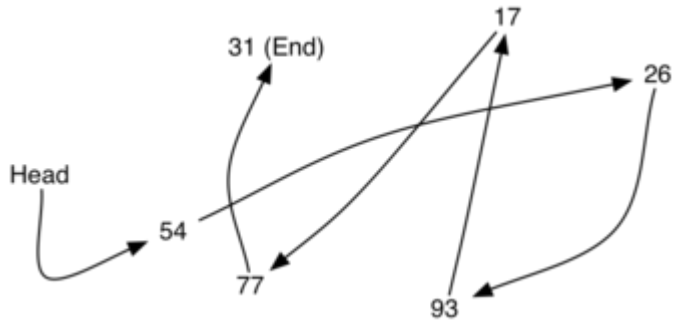
```
class Node:
    def __init__(self,initdata):
        self.data = initdata
        self.next = None

    def getData(self):
        return self.data

    def getNext(self):
        return self.next

    def setData(self,newdata):
        self.data = newdata

    def setNext(self,newnext):
         self.next = newnext
```

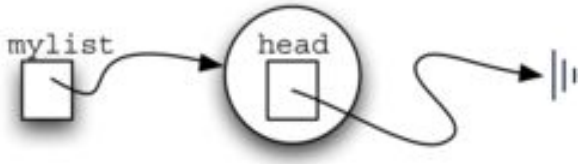


Implementando Listas Ligadas

```
class UnorderedList:
    def __init__(self):
        self.head = None

    def isEmpty(self):
        return self.head == None

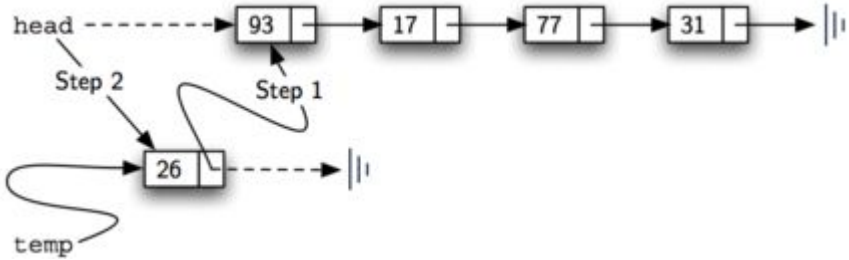
...
```



Implementando Listas Ligadas

...

```
    def add(self, item):
        temp = Node(item)
        temp.setNext(self.head)
        self.head = temp
```



Implementando Listas Ligadas

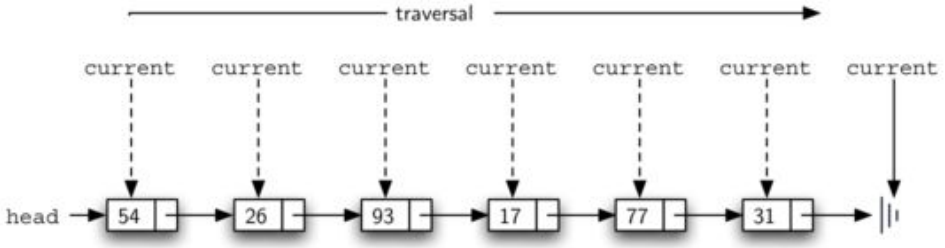
```
...
    def append(self, item):
        temp = Node(item)
        if self.isEmpty():
            self.head = temp
        else:
            current = self.head
            while current.getNext() is not None:
                current = current.getNext()
            current.setNext(temp)
```

Implementando Listas Ligadas

```
...
    def size(self):
        current = self.head
        count = 0
        while current != None:
            count = count + 1
            current = current.getNext()
        return count
```

Implementando Listas Ligadas

```
...
    def size(self):
        current = self.head
        count = 0
        while current != None:
            count = count + 1
            current = current.getNext()
        return count
```

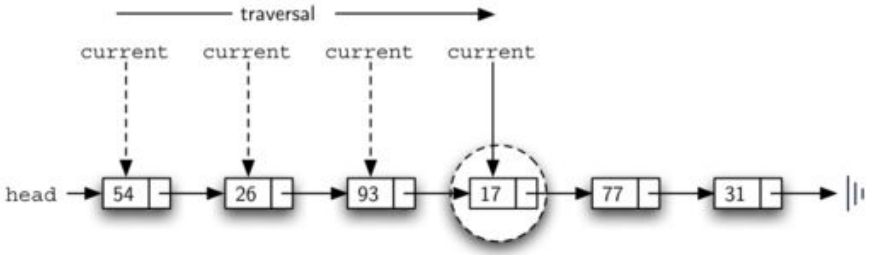


Implementando Listas Ligadas

```
...
    def search(self,item):
        current = self.head
        found = False
        while current != None and not found:
            if current.getData() == item:
                found = True
            else:
                current = current.getNext()
        return found
```

Implementando Listas Ligadas

```
...
    def search(self,item):
        current = self.head
        found = False
        while current != None and not found:
            if current.getData() == item:
                found = True
            else:
                current = current.getNext()
        return found
```



Implementando Listas Ligadas

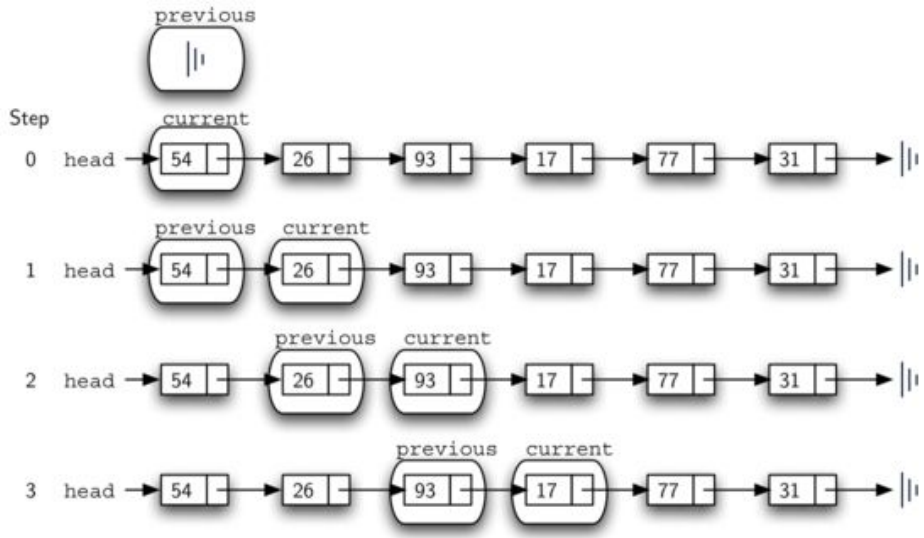
```
...

    def remove(self,item):
        current = self.head
        previous = None
        found = False
        while not found:
            if current.getData() == item:
                found = True
            else:
                previous = current
                current = current.getNext()
        if previous == None:
            self.head = current.getNext()
        else:
            previous.setNext(current.getNext())
```

Implementando Listas Ligadas

```
...

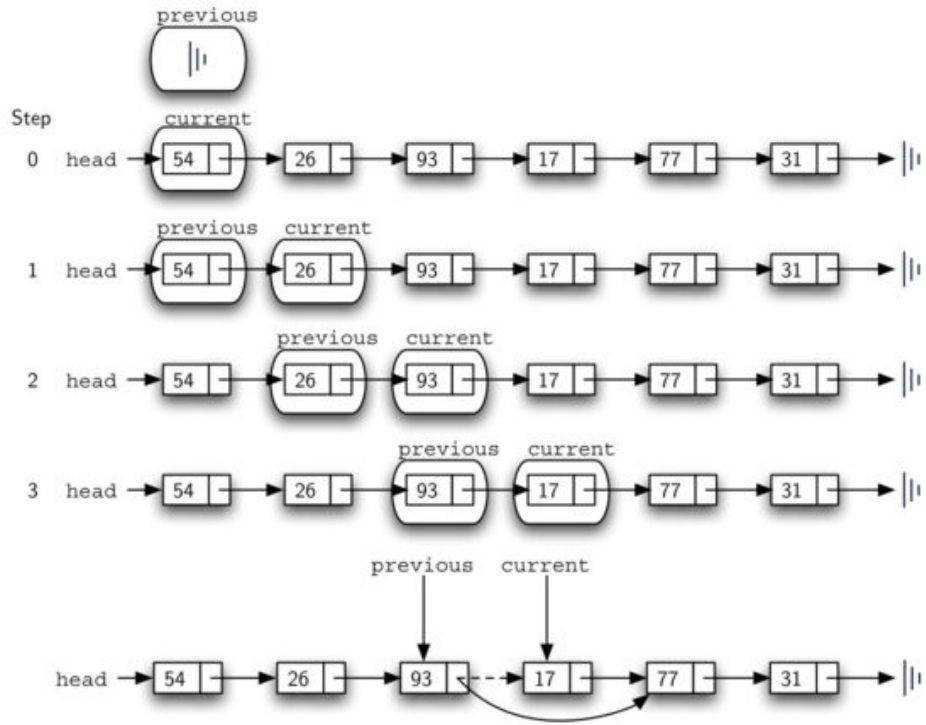
    def remove(self,item):
        current = self.head
        previous = None
        found = False
        while not found:
            if current.getData() == item:
                found = True
            else:
                previous = current
                current = current.getNext()
        if previous == None:
            self.head = current.getNext()
        else:
            previous.setNext(current.getNext())
```



Implementando Listas Ligadas

```
...

    def remove(self,item):
        current = self.head
        previous = None
        found = False
        while not found:
            if current.getData() == item:
                found = True
            else:
                previous = current
                current = current.getNext()
        if previous == None:
            self.head = current.getNext()
        else:
            previous.setNext(current.getNext())
```



Implementando Listas Ligadas

```
...

    def index(self, item):
        pass

    def insert(self, pos, item):
        pass

    def pop(self, pos=None):
        pass
```

Resumo

- Deques são estruturas de dados que permitem um comportamento híbrido de pilhas ou filas.
- Listas são coleções de itens em que cada item possui uma posição relativa.
- Uma implementação de lista encadeada mantém a ordem lógica sem requisitos de armazenamento físico.